

E-Commerce Integration Portfolio - Interview Prep Guide

Purpose: Prepare for interview questions about e-commerce platform integrations, payment processing, and fintech engineering patterns — relevant to work at companies like Shipt integrating with retailers and payment providers.

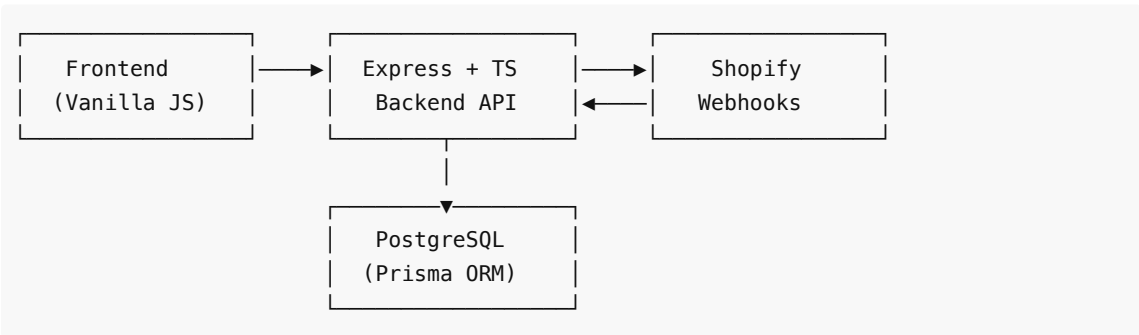
Project Overview: Two Complementary E-Commerce Systems

Project	Payment Provider	Focus Area
StyleHub (CryptoLists/shop)	Shopify Payments	Headless e-commerce, webhook reliability, DLQ
EcomSync	Stripe Checkout	Session-based checkout, React frontend

Both projects demonstrate **production-grade patterns** for handling payments, order management, and external platform integrations.

1. StyleHub — Headless E-Commerce with Shopify

Architecture



Key Technical Decisions

1. HMAC Webhook Verification

```
// Using timing-safe comparison to prevent timing attacks
const isValid = crypto.timingSafeEqual(
  Buffer.from(hmacHeader),
  Buffer.from(calculatedHmac)
);
```

Why it matters: Prevents attackers from forging webhook events. At Shipt, this prevents fake order completions or payment confirmations from malicious actors.

2. Dead Letter Queue (DLQ) Pattern

Webhook → Attempt 1 → Fail → Retry (exp backoff) → ... → DLQ

- Webhooks stored in DB on receive (**RECEIVED** status)
- Background worker processes asynchronously
- Failed events retry with exponential backoff
- After max attempts, moved to **DEAD_LETTER** for manual review

Why it matters: Ensures no lost orders even if downstream systems are temporarily unavailable.

3. Idempotency Service

```
// Prevents duplicate order creation on retries
const existing = await prisma.idempotencyRecord.findUnique({
  where: { key: idempotencyKey }
});
if (existing?.status === 'COMPLETED') {
  return existing.response; // Return cached response
}
```

Why it matters: If a customer's payment succeeds but the confirmation request fails, the retry won't create a duplicate order or double-charge.

4. Database Schema (9 Models)

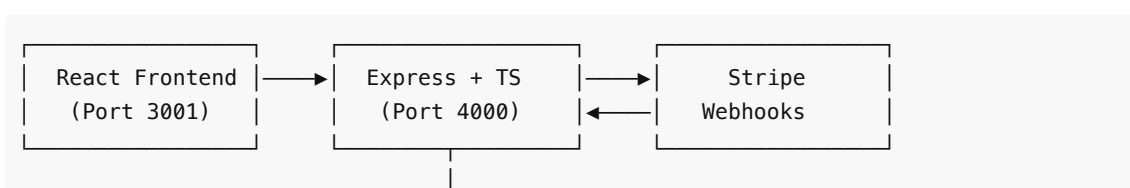
Model	Purpose
Product, ProductVariant	Synced from Shopify catalog
Order, OrderItem	Cart → Order with line items
Transaction	Payment records with idempotency keys
LedgerEntry	Double-entry bookkeeping for auditing
WebhookEvent	DLQ tracking for webhook reliability
IdempotencyRecord	Duplicate request prevention

Frontend: Login-Free Guest Checkout

- **localStorage cart** — no account required
- **Email capture at checkout** — for order confirmation only
- **Variant selection** — size/color before add-to-cart

2. EcomSync — Stripe Payment Integration

Architecture



PostgreSQL
(Prisma ORM)

Key Technical Decisions

1. Stripe Checkout Sessions

- Create session with line items server-side
- Redirect customer to Stripe's hosted checkout page
- No PCI compliance burden (Stripe handles card data)

```
const session = await stripe.checkout.sessions.create({
  line_items: cartItems.map(item => ({
    price_data: { ... },
    quantity: item.quantity
  })),
  success_url: `${domain}/success?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${domain}/cancel`
});
```

2. Webhook-Based Fulfillment (Decoupled)

Critical Pattern: Don't fulfill orders on API response — fulfill on webhook!

- ✗ Wrong: Create order → Stripe payment → Fulfill immediately
- ✓ Right: Create order → Stripe payment → Wait for webhook → Fulfill

Why? The payment could fail after your API returns success, or the customer could close the browser mid-payment.

3. Idempotency for Payments

```
await stripe.paymentIntents.create({
  amount: totalInCents,
  currency: 'usd'
}, {
  idempotencyKey: `order-${orderId}-${Date.now()}`
});
```

4. Order State Machine

PENDING → PROCESSING → COMPLETED
↓ ↓
CANCELLED REFUNDED

Interview Talking Points

"Tell me about a project where you integrated with external payment systems"

"I built two e-commerce systems integrating with Shopify and Stripe. The key challenge was **reliability** — what happens if Stripe confirms payment but my server crashes before updating the database?"

I solved this with:

1. **Idempotency keys** — so retries don't double-charge customers
2. **Webhook-based fulfillment** — orders only fulfill after confirmed webhook, not on API response
3. **Dead Letter Queue** — failed webhooks retry with exponential backoff, then move to DLQ for manual review

This pattern ensures **exactly-once processing** even in failure scenarios."

"How do you handle webhook security?"

"I implemented HMAC signature verification for Shopify webhooks. The key points are:

1. Capture the **raw request body** before JSON parsing
2. Compute HMAC-SHA256 with the shared secret
3. Use **timing-safe comparison** to prevent timing attacks
4. Reject any request with missing or invalid signatures

This prevents attackers from forging webhook events like fake payment confirmations."

"How do you prevent duplicate orders or charges?"

"I implemented an idempotency service pattern:

1. Each request includes a unique idempotency key (usually order ID + timestamp)
2. Before processing, check if we've seen this key before
3. If completed, return the cached response
4. If processing, wait or reject (prevent race conditions)
5. Store the final response for future retries

This is critical for payment systems where network failures during confirmation are common."

"Walk me through the checkout flow"

"The flow differs slightly between Shopify and Stripe:

Stripe: Cart → Create checkout session (server) → Redirect to Stripe hosted page → Payment → Webhook → Update order status → Show success

Shopify: Cart → Create order (PENDING) → Redirect to Shopify checkout → Payment → ORDERS_PAID webhook → Update order (COMPLETED) → Background fulfillment

In both cases, the key is **decoupled fulfillment** — we don't assume payment succeeded until we get the webhook confirmation."

Relevance to Shipt

Project Pattern	Shipt Equivalent
Retailer platform integration	Integrating with Target, Publix, Costco catalogs

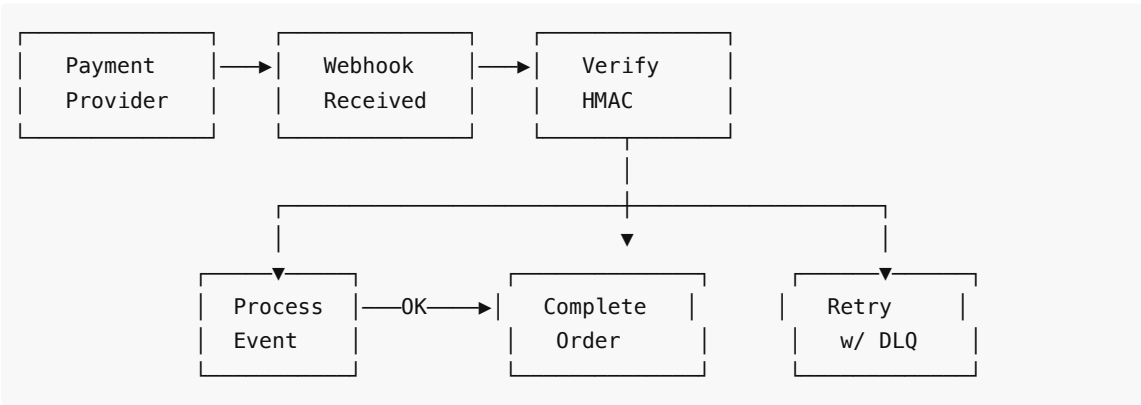
Payment webhook handling	Stripe payment confirmations
Idempotency for orders	Preventing duplicate shopper assignments
DLQ for reliability	Handling retailer API outages gracefully
Order state machine	Order lifecycle (placed → shipped → delivered)

Tech Stack Summary

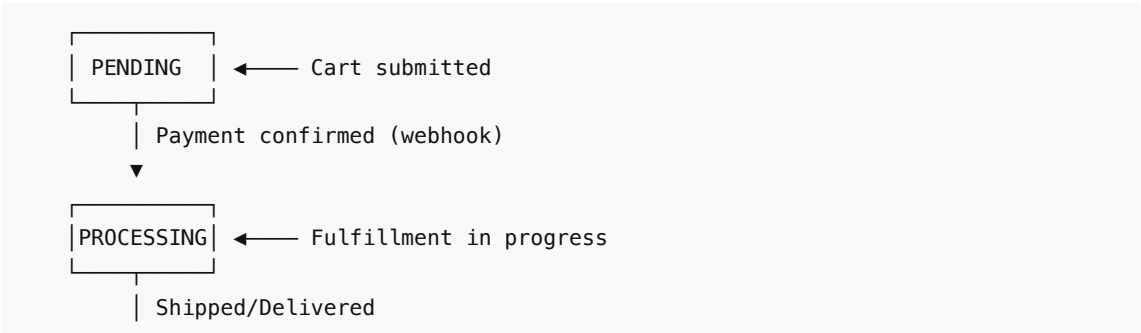
Layer	StyleHub (Shopify)	EcomSync (Stripe)
Frontend	Vanilla JS + CSS	React + React-Bootstrap
Backend	Express + TypeScript	Express + TypeScript
Database	PostgreSQL + Prisma	PostgreSQL + Prisma
Payments	Shopify Checkout	Stripe Checkout Sessions
Security	HMAC webhooks	HMAC webhooks
Reliability	DLQ + Idempotency	Idempotency

Quick Reference: Architecture Diagrams

Webhook Flow



Order State Machine



▼
COMPLETED

← Done

(Any state can → CANCELLED or REFUNDED)

Files to Reference

StyleHub (CryptoLists)

- [shop.html](#) — Frontend
- [shop.js](#) — Cart logic
- [server/src/index.ts](#) — Express server
- [server/prisma/schema.prisma](#) — Database models
- [server/src/middleware/shopify-webhook-verifier.ts](#) — HMAC verification

EcomSync

- [README.md](#) — Project overview
- [server/](#) — Backend with Stripe
- [store/](#) — React frontend

Last updated: February 2, 2026